Computation-normal iPRA

A six-rule calculus, a fresh-zero lemma,
and a smaller validated checker

Research prepared for Vladimir Reshetnikov

11 November 2026 (Pacific time)

Abstract

We investigate a simplification of the current ConciseIPRA proposal at a fixed repository snapshot. There are two separate results. First, both iterator computation axioms are replaced by computation-normal syntax and reflexivity. We prove that a six-rule calculus results and give the round trips required for bi-interpretability with intuitionistic primitive recursive arithmetic. The main new completeness argument is a fresh-zero lemma: a locally introduced variable provably equal to zero replaces the closed zero alias that full normalization destroys.

Second, two existing validation ideas are composed without changing the arithmetic calculus. The complete checker has 860 normalized source bytes, 377 significant tokens and 433 AST nodes, versus 870, 383 and 442 for its matched public Eq/Q/D baseline. Maximum-level gzip is 508 bytes, versus 507. The acceptance contract is preserved.

These results do not establish an overall simplest axiomatization. The repository already contains a six-rule zero-normal presentation and a 516-byte checker with a different arity-envelope interface. Fair comparisons show that full normalization does not robustly reduce complete checker size; it also introduces a total-recursive, non-primitive-recursive uniform normalizer. We recommend the small validation reduction as an implementation improvement and retain the new calculus as a proved mathematical alternative, with its costs explicit.

1 Result and comparison contract

The requested objective is a simpler theory that is both equiconsistent and bi-interpretable with full first-order iPRA. Preserving only its equational fragment would not suffice. We use the size of a complete proof-checker implementation as the operative complexity criterion, and record gzip at compression level 9 as an informational measurement.

The baseline is repository commit `d9383325b18d349b8bfdc54ef564c5784cc029ec`, which introduces `NativeZeroNormal.lean`. Its UTC commit time is 12 November 2026, 03:01:27, or 11 November in Pacific time. Comparisons are pinned to that snapshot rather than to a moving branch [1].

The snapshot distinguishes the eight-rule native kernel, the seven-rule single-introduction variant, the adopted seven-rule zero-normal kernel, and a delivered six-rule zero-normal variant. It also contains much smaller arity-coded Python checkers. Their validation contracts differ. The 516-byte compact source is an implementation refinement separate from its package's alternative recursor theory [2].

Established results. The calculus $\mathcal{N}_6$ below has six rule kinds, with reflexivity in place of computation equations. Its normalized sequents are exactly those of the native calculus,

and both interpretation composites are provably the identity. The fresh-zero lemma makes its single introduction complete despite the loss of closed nonliteral zero aliases. Independently, deleting a duplicated public-syntax scope check produces a smaller complete checker for the unchanged existing theory.

Limits of the claim. Neither construction is a proof of global minimality. The 860-byte program does not beat the different 516-byte interface. Full normalization simplifies the explicit axiom table but does not win robustly in matched complete-checker measurements. No new Lean or Rocq formalization is claimed: the extension consists of written proofs, executable checkers, concrete certificates, and independent review. The native/iPRA bridge is inherited from the pinned repository.

2 The inherited arithmetic and logic

2.1 The reference iPRA

We use the repository's first-order intuitionistic presentation. There is one numerical sort, actual equality, zero, successor, and one function symbol for each primitive-recursive definition built from zero, successor, projections, composition and primitive recursion. The nonlogical axioms are the defining equations, successor injectivity, $Sx = 0 \to \bot$, and induction for quantifier-free formulas, with parameters and side assumptions. The ambient logic is intuitionistic first-order natural deduction with equality. The primitive-recursive function family is an effective schema, not a hidden finite table.

2.2 The raw native calculus

Raw terms are

$$t ::= x \mid 0 \mid S(t) \mid I(c, a;\, u.h).$$

Only u is bound in h, not in the counter c or seed a. The computation equations are

$$I(0, a;\, u.h) = a, \tag{1}$$
$$I(Sc, a;\, u.h) = h[I(c, a;\, u.h)/u]. \tag{2}$$

Formulas have the grammar

$$A ::= (s = t) \mid Q_y B \mid D_{x,y} B.$$

The designated contradiction is $0 = S0$. Their ordinary intuitionistic readings are

$$(Q_y B)^\circ := B^\circ(0) \to \forall y\, B^\circ(y), \tag{3}$$
$$(D_{x,y} B)^\circ := \exists x\, (B^\circ(x, 0) \to \forall y\, B^\circ(x, y)). \tag{4}$$

No classical principle is implicit in these readings.

The raw reference calculus $\mathcal{R}$ is `NPrf`: hypothesis, the two computation equations, crossed one-term transport, Q elimination, D elimination, D introduction by cases and Q introduction by equational progress. Its two introductions are

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)} \qquad \frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_y E(y)},$$

where z is fresh and E is a literal equation.

2.3 The inherited bi-interpretation

The intermediate ordinary theory T_{eq} uses intuitionistic logic with equality, the iterator equations, coverage $\forall x(x = 0 \vee \exists y\, x = Sy)$, separation $0 = S0 \to \bot$, and equation induction with side assumptions. The pinned development proves that $\mathcal{R}$, T_{eq} and iPRA are bi-interpretable, with whole numerical domains and the identity graph $x = y$ on the composites.

The final results are named as follows:

Equivalence: `native_iff_pra`, `pra_iff_native`.
Composites: `native_roundTrip`, `pra_native_roundTrip`.
Consistency: `native_inconsistent_iff`.

Their definitions are in `NativeBiInterpretation.lean`. The archive includes that source for precise attribution; the dependency chain remains in the pinned repository. We do not replace it with a new unproved representation assertion.

The bridge derives predecessor, reflection of equality into a term, quantifier-free induction, pairing, and a compiler for general primitive recursion. The logical layer defines $\bot, \to, \wedge, \vee, \exists, \forall$ by selector and gate formulas and proves all their intuitionistic natural-deduction rules. Consequently a faithful new presentation of $\mathcal{R}$ retains full first-order intuitionistic arithmetic.

3 Computation-normal syntax

Orient (1) and (2) from left to right, allowing reduction everywhere, including under iterator binders.

Definition 3.1. The normal syntax, with scopes understood, is

$$\begin{aligned} n &::= x \mid I(n, a;\, u.h), \\ a, h &::= n \mid 0 \mid S(a). \end{aligned}$$

The first line describes neutral terms. Every iterator counter has a neutral root, and all subterms are normal. Formula constructors are unchanged; their numerical terms must be normal.

For example, $I(x, 0;\, u.0)$ is normal, although its intended value is always zero. In contrast, $I(Sx, 0;\, u.0)$ normalizes to 0. Computation normalization does not decide arithmetic equality.

Write $\mathrm{nf}(t)$ for the normal form. Substitution in the new calculus is always *normal substitution*:

$$t\langle s/x\rangle := \mathrm{nf}(t[s/x]).$$

We use ordinary substitution notation in rules with this convention in force. Formula normalization changes terms only and preserves the equation/Q/D tree.

Lemma 3.2 (External normalization)**.** *Every finite well-scoped raw term has a unique normal form. Normalization is idempotent, preserves scope, and satisfies*

$$\mathrm{nf}(t\sigma) = \mathrm{nf}(\mathrm{nf}(t)\,(\mathrm{nf}(\sigma))). \tag{N}$$

Proof. For termination, embed the syntax in Gödel's System T. All numerical variables have base type $\mathbb{N}$, and the iterator maps to

$$\mathrm{Rec}_{\mathbb{N}}(a, \lambda k^{\mathbb{N}}.\lambda u^{\mathbb{N}}.h, c),$$

with k fresh and unused. A source zero contraction becomes a recursor contraction; a successor contraction becomes a recursor contraction followed by two beta contractions. Translation

respects contexts and capture-avoiding substitution. Thus an infinite source reduction would give an infinite System-T reduction, contrary to strong normalization. A primary reference with a mechanized proof is Urciuoli [5]. System T is used in the metatheory, not added to the arithmetic being presented.

For confluence, use parallel reduction. The two left-hand sides have different constructor patterns and are left-linear. Reductions in counter, seed and body commute with root contraction after taking parallel reducts. The duplicated terms in a successor contraction are reduced in parallel. Induction proves the substitution property $h \Rightarrow h'$, $r \Rightarrow r'$ implies $h[r/u] \Rightarrow h'[r'/u]$, and then the diamond property for parallel developments. Termination and confluence yield unique normal forms. Stability of reduction under substitution proves (N); idempotence and scope preservation follow. □

This is an external termination proof. It asserts no iPRA-internal uniform evaluator theorem. Injective variable renaming preserves neutral roots and normal syntax. In particular, eigenvariable shifts commute with normalization. Equation (N) supplies the identity and associativity laws of normal substitution.

Lemma 3.3 (Finite conversion soundness in $\mathcal{R}$)**.** *For every individual raw term t, $\mathcal{R}$ proves $t = \mathrm{nf}(t)$. Every native formula is interderivable with its normalization, over all its free parameters.*

Proof. Root reductions are computation axioms; reductions in counter and seed use ordinary congruence. Reduction in a bound update body needs more care: the iterator is not an ordinary ternary function of three numbers.

If $h(u) = h'(u)$ is provable with u fresh, raw equation induction on c proves $I(c, a; u.h) = I(c, a; u.h')$. At zero both sides are a. At a successor, unfold, specialize the pointwise body equality at the recursive value, and use the induction hypothesis inside the other body. This is the required body-congruence lemma. Lift and compose the finitely many reduction steps. The established raw Q and D congruences then lift term equalities to formula interderivability. □

The body-congruence argument uses induction in the already established raw calculus. It does not assume the new calculus's target theorem.

4 The complete six-rule axiomatization

Contexts are finite lists of normal formulas, and hypotheses may be reused. Every substitution below is normal substitution. In DE, b is fresh for Γ, A, and B; in DI, z is fresh for Γ, B, and the witness t. These six kinds are the entire primitive calculus $\mathcal{N}_6$.

Hypothesis and reflexivity.

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ (\textsc{Hyp}), \qquad \frac{}{\Gamma \vdash t = t}\ (\textsc{Refl}).$$

The reflexivity term is any well-scoped normal term.

Crossed transport in one term.

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)}\ (\textsc{Tr}).$$

The normal template h has one displayed argument in addition to ambient parameters; it may use the argument repeatedly or not at all.

Closure elimination.

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ (\mathrm{QE}).$$

Witness elimination.

$$\frac{\Gamma \vdash D_{x,y} B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash A}{\Gamma \vdash A}\ (\mathrm{DE}), \qquad b \text{ fresh.}$$

Single witness introduction.

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y} B(x,y)}\ (\mathrm{DI}), \qquad z \text{ fresh,}$$

where

$$\eta = \begin{cases} z, & B \text{ is literally an equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases}$$

The guard is syntactic, not a test whether t is provably zero.

There are no other primitive logical axioms, equality laws, successor axioms, induction schemas, structural rules, or computation equations. The computation clauses belong to the *specified substitution algorithm*; its implementation is fully counted. Arithmetic computation has not disappeared from the trusted machinery.

4.1 Ordinary soundness of introduction

In case mode, the premise gives $B(t,0) \to B(t,Sz)$ for arbitrary z. Coverage, using $B(t,0)$ in the zero case, yields $B(t,0) \to \forall y B(t,y)$; existential introduction gives the reading of D. In progress mode the matrix is a literal equation of primitive-recursive terms, so quantifier-free induction gives exactly the needed closure. There is no induction on arbitrary compound Q/D formulas.

5 The fresh-zero completeness lemma

Lemma 5.1. *Weakening, injective renaming and cut are admissible for $\mathcal{N}_6$, without first assuming arbitrary numerical substitution.*

Proof. Use induction on derivations. Injective renaming preserves normality, literal zero, the matrix root, and freshness. For cut, under each eigenvariable rule first rename and weaken the inserted proof. No guard depends on membership or multiplicity of context formulas, so the ordinary substitution of derivations for hypotheses applies. □

Arbitrary numerical substitution can turn a syntactically nonzero witness into literal zero, changing the selected premise. A direct induction claiming substitution admissibility for the single introduction would therefore be invalid. We derive it later through a stable equivalent basis.

5.1 Atomic equality reasoning

Symmetry of $s = t$ is one Tr step with major $s = s$, identity template and second premise $s = t$. Congruence uses major $s = t$ and second premise $h(s) = h(s)$. For transitivity, reverse $s = t$ to $t = s$ and transport it along $t = u$, obtaining $s = u$.

Two applications give two-sided atomic transport: from $s = t$ and $a(s) = b(s)$ derive $b(s) = a(t)$ and then $a(t) = b(t)$. Equation (N) ensures the normalized template instances match exactly.

5.2 The lost zero alias

The old zero-normal six-rule argument can use $I(S0, 0; u.0)$ as a closed, nonliteral term provably zero. Full normalization erases that distinction. Indeed every closed full-normal term is a numeral: following the counter chain of a neutral term would otherwise eventually require a free variable.

Lemma 5.2 (A fresh name for zero)**.** *Put* $Z(x, y) \equiv I(y, 0; u.x) = 0$. *Then* $\mathcal{N}_6 \vdash D_{x,y}Z(x, y)$. *In its* DE *scope, the fresh witness* b *satisfies* $b = 0$.

Proof. Introduce DZ at witness 0. The matrix is an equation, so the rule selects progress mode. Its successor target normalizes to $0 = 0$, proved by reflexivity without using the progress hypothesis.

In the elimination scope assume $Q_yZ(b, y)$. The base $Z(b, 0)$ normalizes to $0 = 0$. Use reflexivity for that base and QE at $S0$. The result $Z(b, S0)$ normalizes to $b = 0$. □

The local witness is a variable, despite being provably zero. It is discharged before the final conclusion; no permanent constant or additional domain element has been introduced.

5.3 Recover the omitted atomic case

Suppose $E(x, y)$ is an equation and a proof is given of

$$\Gamma, E(0, 0) \vdash E(0, Sz), \qquad z \text{ fresh.} \tag{C0}$$

The desired $D_{x,y}E(x, y)$ is not a primitive case introduction at witness zero, because the guard would demand progress.

Open Lemma 5.2, obtaining fresh b and $b = 0$. Under $E(b, 0)$, atomic transport gives $E(0, 0)$. Cut into (C0), then transport the resulting $E(0, Sz)$ back along $0 = b$. This gives $E(b, Sz)$. Apply primitive DI at witness b, where case mode is selected. Its conclusion DE and the context do not mention b, so DE closes the fresh-zero scope.

Only atomic transport, weakening, injective renaming and cut have been used. Full formula transport and arbitrary term substitution were not premises of this recovery.

6 Equivalence and both interpretation composites

Let $\mathcal{N}_7$ have the same normal syntax and five shared rules, but replace the single DI by unrestricted D introduction by cases and Q introduction by equational progress:

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)} \qquad \frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_yE(y)}.$$

The equation restriction and freshness conditions remain in force.

Theorem 6.1. $\mathcal{N}_6$ *and* $\mathcal{N}_7$ *derive the same normal sequents.*

Proof. All case instances except the equation/witness-zero case are already primitive in $\mathcal{N}_6$; Section 5 derives that last case. For progress, add a dummy witness x to $E(y)$ and apply DI at zero. Eliminate $D_{x,y}E(y)$; the local hypothesis $Q_yE(y)$ gives the conclusion.

Conversely, case-mode DI is the case rule of $\mathcal{N}_7$. For progress-mode DI, use equational progress to obtain $Q_yE(0, y)$. Under $E(0, 0)$, QE gives $E(0, Sz)$. The case rule at witness zero gives $D_{x,y}E(x, y)$. □

Every $\mathcal{N}_7$ rule is stable under arbitrary normal substitution. Its substitution induction is valid and transfers the admissibility result to $\mathcal{N}_6$ through this equivalence.

Theorem 6.2 (Faithfulness to the raw kernel)**.** *For every raw native context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{R}} A \quad \Longleftrightarrow \quad \mathrm{nf}(\Gamma) \vdash_{\mathcal{N}_6} \mathrm{nf}(A). \tag{F}$$

The translations preserve free variables.

Proof. For the forward direction normalize every formula in a raw derivation, translating into $\mathcal{N}_7$. Both computation equations become reflexivity. Transport, closure elimination and the two introductions commute with normal substitution by (N). Witness elimination commutes with fresh renaming. Induction on the raw proof gives the result in $\mathcal{N}_7$, and Theorem 6.1 gives $\mathcal{N}_6$.

For the converse first translate $\mathcal{N}_6$ into $\mathcal{N}_7$. Include each normal term as a raw expression. Reflexivity is a raw theorem. Each normalized substitution instance is raw-provably equal or interderivable with the corresponding raw instance, by finite conversion soundness. The raw calculus's already established equality and formula transport then realize Tr, QE and the case rule. Raw equational induction realizes progress; witness elimination is unchanged up to renaming. The resulting raw proof of $\mathrm{nf}(A)$ from $\mathrm{nf}(\Gamma)$ converts back to the original sequent using the formula normalization equivalences. □

The reverse translation uses properties of the *raw* calculus that precede the new result. This is not an appeal to the target theorem inside its own proof.

6.1 The round trips contain more than theorem preservation

Let N normalize raw syntax and let J include normal syntax into raw syntax. For a normal formula A, $N(J(A)) = A$ literally. For each raw term t, the raw theory proves $J(N(t)) = t$; for each raw formula it proves the corresponding interderivability over all free parameters.

Compose J with the existing native-to-iPRA interpretation, and compose the existing iPRA-to-native interpretation with N. The inherited round trips and the normalization round trips give:

Corollary 6.3. *The ordinary realization of* $\mathcal{N}_6$ *is bi-interpretable with* iPRA*. Both numerical domains are whole, equality is actual equality, and each composite domain isomorphism is defined by* $x = y$*.*

The normal language is a computational presentation: terms are canonical representatives and substitution computes representatives. Its ordinary realization reads them as raw iterator terms. This does not change the domain into codes for standard numbers or discard nonstandard model elements.

Corollary 6.4 (Equiconsistency)**.**

$$\mathcal{N}_6 \vdash 0 = S0 \quad \Longleftrightarrow \quad \mathsf{iPRA} \vdash \bot.$$

Proof. The contradiction is already normal. Apply (F), then use the inherited consistency theorem to identify raw contradiction with iPRA's contradiction. □

These are ordinary external mathematical statements. They do not silently assert a primitive-recursive uniform proof translator or an iPRA-internal normalization theorem.

7 The cost of full normalization

Proposition 7.1. *A uniform full normalizer over codes for all iterator terms cannot be primitive recursive.*

Proof. Suppose it were. Enumerate finite well-formed unary iterator terms, replacing malformed descriptions by the constant-zero program. Syntax checking, forming a closed instance at a numeral, and reading the final numeral are primitive-recursive operations. The inherited representation theorem says these terms represent every primitive-recursive unary function. The assumed normalizer would therefore give a primitive-recursive universal evaluator $U(e, n)$ for those functions.

Then $d(n) = U(n, n) + 1$ would itself have a program code e_d. At $n = e_d$ this gives $U(e_d, e_d) = U(e_d, e_d) + 1$, a contradiction. □

Each fixed term still defines a primitive-recursive function. The obstruction is to a single uniform evaluator over all descriptions. This is why eliminating a computation rule does not eliminate its cost from the trusted implementation.

The mathematical unbounded checker is total recursive on well-formed syntax. Like the baseline recursive Python programs, physical execution is subject to memory and recursion limits. Source-size measurements do not imply efficient checking. Every closed full-normal term is an expanded numeral, so short closed expressions can have enormous normal forms.

7.1 A fuelled certificate interface

The archive includes a bounded version. A certificate supplies a nonnegative integer fuel, and each successor contraction spends one unit. Exhaustion rejects. Zero contractions and the intervening walks are structural.

The corresponding abstract verification relation is primitive recursive: a bound on non-structural contractions yields primitive-recursive bounds on intermediate syntax growth and structural work. Every successful unbounded proof has a finite execution and therefore some sufficient fuel. Existentially quantifying supplied fuel preserves the external theorem set. The extra checking code is included in its measured size.

8 The smaller checker for the existing theory

The directly recommended implementation change is independent of $\mathcal{N}_6$. It preserves the existing arithmetic and logical calculus, proof packets, and kernel function `c`. The matched file is

`validated_presentation/code/native_typed_full.py`.

Only its public syntax validator `v` changes.

8.1 Exact input and acceptance contract

The input is a JSON array. The final element is the closed public goal; earlier elements are proof packets consumed from the end. An unused prefix is allowed. Acceptance means exit status zero and exactly `ACCEPT` followed by a newline.

A lexical gate checks the *entire* certificate for integer/list data. It rejects booleans, floats, strings, nulls and mappings, even in an unused prefix. The public validator checks canonical Eq/Q/D goal syntax. Intermediate resources use the inherited arity-envelope interpretation. Thus this contract is not the tagged typing of every packet field used by the readable checker in the earlier sections.

Before reading any proof packet, the kernel performs an exact scope traversal. The old public validator separately checks an upper bound on each variable slot. That second copy of the upper-bound test is redundant.

8.2 The invariant justifying the deletion

The old validator carries integer depth d. In formula mode, arity two changes to term mode, while arities one and three preserve formula mode. In term mode, valid arities one and three preserve term mode. The new validator needs only the Boolean $\delta = d \bmod 2$. Its update is `d or len(a) == 2`.

The old scalar check allows zero or an odd slot satisfying $2 < a < d$. The new check allows zero or an odd slot satisfying $a > 2$. Parity and the lower bound remain. The upper bound is delegated to the kernel's unavoidable initial scope opener.

Theorem 8.1 (Same successful traces)**.** *The 860-byte checker and its 870-byte baseline accept exactly the same certificates, under their common mathematical execution semantics.*

Proof. On integer/list data, induction on shape shows that Boolean mode equals the old depth parity wherever shape is valid. All old checks remain except the scalar upper bound. Hence every old-valid goal is new-valid.

Suppose a newly admitted goal has an out-of-scope slot. Before any packet is consumed, the unchanged kernel calls its opener `i(a)`. At that slot the condition `k <= a < d` fails, and the opener attempts to use its absent substitution argument. In the initial call that argument is the function `c`, not an integer/list term; the attempted term operation raises an exception. No proof packet can prevent or repair this failure.

Therefore every newly admitted goal violating the old bound is still rejected before proof processing. All other inputs execute the same kernel with the same packets. The lexical gate, arity and parity checks, context rules and acceptance output are unchanged. □

This composes ideas already present in `scope_kernel` and `validated_presentation`. It is an implementation improvement for an unchanged axiomatization, not a new arithmetic axiom deletion.

9 Measurements and executable verification

9.1 Whole-source measurement

Each row counts a complete standalone program, including input loading, validation, kernel and output. Tests, certificate producers and measurement scripts are not imported by a checker. Source is normalized with `ast.unparse(ast.parse(source))` and one final newline. Bytes are UTF-8 bytes of that source. Significant tokens exclude comments, encoding, endmarker and layout. AST nodes are counted by `ast.walk`. Gzip is `gzip.compress(source, compresslevel=9, mtime=0)`, including its wrapper. The reference runtime is CPython 3.12.14 and zlib 1.3.2.

Complete program	Bytes	Tokens	AST	Gzip 9
Compact arity envelope	516	265	306	332
Public Eq/Q/D baseline	870	383	442	507
Public Eq/Q/D scope reuse	860	377	433	508
Zero-normal original	3095	1013	1075	977
Zero-normal term certificate	3029	991	1049	971
Full-normal initial	3048	993	1046	965
Full-normal neutral guard	3044	992	1046	964
Full-normal explicit fuel	3204	1024	1086	1003
Zero-normal goal directed	2982	972	1004	965
Full-normal goal directed	3063	982	1008	961
Full goal directed, fuel	3189	1010	1044	1001

The 860-byte program improves its matched baseline by ten bytes (about 1.15%), six tokens and nine AST nodes. Its gzip is one byte larger. The different 516-byte arity-envelope program remains smaller in absolute size; we do not call the new result a repository-wide minimum.

9.2 The initially apparent full-normal saving does not survive

The original readable zero-normal checker is 3,095 normalized bytes. A full-normal version with a term-shaped reflexivity certificate is 3,048, an apparent 47-byte gain. But the old computation certificate can equally be represented by its iterator term. That change reduces the old checker to 3,029 bytes without changing its calculus. The matched full-normal checker with a neutral-counter test is 3,044 bytes: fifteen bytes and one token larger, although three AST nodes and seven gzip bytes smaller.

A second comparison uses goal-directed programs. Both share an empty leaf certificate and inspect the supplied validated goal. The zero-normal program is 2,982 bytes, full-normal 3,063, and fuelled full-normal 3,189. Thus the simpler explicit axiom inventory does not by itself give a smaller complete checker.

The negative comparison is part of the result: certificate ergonomics must not be advertised as a mathematical breakthrough.

9.3 Verification performed

The recommended validator change was checked on 68 fixtures through both old and new programs, under ordinary and optimized Python: 272 complete process runs, all matching the expected outcomes. A separate 9,000-case comparison covers canonical, malformed and out-of-scope preflights, with no difference.

For full normalization the suite checks a closed fresh-zero certificate and a recovered atomic case-at-zero proof that uses its hypothesis. The latter has context $a = b$ and matrix $I(y, a;\ u.b) = I(y, x;\ u.0)$. It also performs 1,000 small normalization, semantic, scope, idempotence and substitution checks with fixed random seed, plus rejections of a false equation, nonnormal goal, Boolean hypothesis index and escaped variable. Goal-directed translations and both sufficient-fuel acceptance and zero-fuel rejection are checked.

These tests exercise substantive failure boundaries; they are not proof-assistant verification. The general mathematical arguments and their finite executable evidence are reported separately.

10 Recommendation

Retain the current arithmetic presentation for immediate repository use, and adopt the 860-byte validator composition if its canonical public Eq/Q/D contract is desired. Its benefit is

modest and measured, and no new arithmetic soundness theorem is needed.

The full-normal calculus is a useful research alternative. Its explicit equational axiom is simple, and the fresh-zero lemma solves a real completeness problem. Its costs are also clear: matched total checker size does not uniformly improve, normalization can be enormous, and the uniform evaluator is not primitive recursive. It should not replace the structural zero-normal kernel solely because its arithmetic rule table looks shorter.

A valuable next formal step is to define full conversion with finite witnesses, prove normalized substitution laws, formalize the fresh-zero and omitted-case derivations, and connect them to `native_iff_pra`. This report does not claim that this Lean development has already been completed.

A Tagged certificate format

The readable candidate uses the following JSON terms and formulas:

Object	Encoding
Variable	`["v", i]`
Zero	`["z"]`
Successor	`["s", t]`
Iterator	`["i", c, a, h]`
Equation	`["=", s, t]`
Closure	`["q", B]`
Paired binder	`["d", B]`

In an iterator body, de Bruijn index zero is the accumulator. In a Q body it is the bound variable. In a $D_{x,y}$ body, index zero is y and index one is x.

A certificate has exactly `variables`, `context`, `goal` and `proof`; the bounded variant also has `fuel`. The first field is the ambient scope size. A normal term itself is its reflexivity certificate. Other nodes are `["hyp", i]`, `["tr", h, p, q]`, `["qe", t, p, q]`, `["de", A, p, q]`, and `["di", B, t, p]`. The context, goal, templates, witnesses and used embedded term/formula fields are explicitly checked.

B Reproduction and inventory

From the extracted package directory run:

```
python3 reproduce.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The reproduction script reruns verification, measures selected complete sources, and regenerates normalized source and deterministic gzip artifacts. It does not fetch a moving repository baseline.

The archive contains this source and PDF, complete checkers, matched controls, proof certificates, tests, machine-readable evidence, source provenance, the independent completeness audit, and a SHA-256 manifest. The source license is preserved.

C The complete recommended checker

The following is the complete 860-byte normalized program. The arity codec is inherited; Section 8 identifies exactly which duplicated validation has been removed.

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <=
     a < d else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or
     (a := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)], k)

def v(a, d=0):
    return d and (not a or (a > 2 and a % 2)) if a * 0 == 0 else (len(a) in (1, 2 + d) or a
    == [a[0]] * 3) and all((v(b, d or len(a) == 2) for b in a))
q = json.load(open(0))
if repr(q).strip('0123456789[], -') or not v((a := q.pop())):
    raise ValueError('The certificate must use integer/list syntax and have a canonical
    closed goal')
q = q.pop
c(a)
print('ACCEPT')
```

References

[1] Vladimir Reshetnikov and ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*. Commit-pinned project directory (snapshot fixed in Section 1). See `README.md` and the modules `Native.lean` and `NativeBiInterpretation.lean` under `Lean/ConciseIPRA/`.

[2] ProveIt research package, *canonical_recursors*. Same snapshot, under `Research/canonical_recursors/compact/`; file `code/checker_compact_zero.py`. The 516-byte source is distinct from that package's alternative recursor theory.

[3] ProveIt research package, *zero_normal*. Same snapshot, `Research/zero_normal/code/zero_six.py`. The adopted normalization layer is `Lean/ConciseIPRA/ZeroNormalization.lean`.

[4] ProveIt research packages, *validated_presentation* and *scope_kernel*. Same snapshot. The matched package is `Research/validated_presentation/`; its source is `code/native_typed_full.py`.

[5] Sebastián Urciuoli. *A Formal Proof of the Strong Normalization Theorem for System T in Agda*. EPTCS 376 (2023), 81–99. DOI: 10.4204/EPTCS.376.8. https://arxiv.org/abs/2303.13258.